

M18 / PROTECTED ANSWERS

M18 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	4
1. Explain why a portfolio pipeline that produces the correct final CSV once can still fail the M18 standard.	4
2. Design an immutable raw-landing identity and explain changed-content collision behavior.	4
3. A source batch has 1,000 rows, 12 invalid business records and a sink failure after 700 writes. Separate quarantine from infrastructure recovery and state ownership.	4
4. Show three source-to-target reconciliation checks for an order pipeline.	4
5. A fact join doubles revenue. Diagnose the likely grain/cardinality problem and prove the repair.	5
6. Explain why an incremental watermark using max(updated_at) can still miss data, and give a safer bounded strategy.	5
7. Define a dbt-style staging/fact/dimension structure and where tests belong.	5
8. Describe an orchestration failure where progress outruns durable output; give the safe repair.	5
9. For Spark, explain how partition count, shuffle and skew evidence change a performance decision.	5
10. Define Bronze/Silver/Gold as contracts rather than folder names.	6
11. An additive schema field appears. Describe compatibility, quarantine/versioning and downstream validation.	6
12. Map a local lakehouse project to Fabric items without claiming a tenant run you did not execute.	6
13. Choose CI checks that block deployment versus observability checks that alert after deployment.	6
14. Write the minimum contents of a data-pipeline runbook.	6
15. Explain what a clean-clone proof establishes and what it does not establish.	7
16. Give a two-minute technical defense structure for a portfolio project.	7

17. Capstone incident: duplicated payment event plus stale inventory CDC change. Explain safe diagnosis, replay and uniqueness/version controls. 7

18. State when Spark/Delta is justified in the capstone and when it is unnecessary complexity. . 7

19. Explain how least privilege and secret boundaries should appear in a Fabric/cloud deployment design. 7

20. List the evidence bundle you would show an interviewer to prove this is your own engineering work. 8

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open after the saved closed-book attempt. Each answer is one acceptable solution shape; alternatives can pass if they prove the same invariant.

1. Explain why a portfolio pipeline that produces the correct final CSV once can still fail the M18 standard.

A one-time correct file proves only one happy-path outcome. M18 also requires reproducibility, declared grain/state, reconciliation, replay/idempotency, failure recovery, changed-input behavior and evidence another engineer can inspect.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

2. Design an immutable raw-landing identity and explain changed-content collision behavior.

A defensible identity includes source + collision-resistant run/object identity plus byte hash/size. Identical replay may resolve to the existing verified object. Same identity with different bytes must fail or create a new immutable identity; it must never overwrite the earlier object.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

3. A source batch has 1,000 rows, 12 invalid business records and a sink failure after 700 writes. Separate quarantine from infrastructure recovery and state ownership.

The 12 business-invalid rows may be durably quarantined under policy. A sink outage is infrastructure failure, so valid unwritten records stay recoverable and progress must not advance past the last durably validated result. After recovery, replay from the last safe state and reconcile accepted + quarantine.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

4. Show three source-to-target reconciliation checks for an order pipeline.

Examples: processed input = accepted + quarantine; unique business key count agrees with target grain; source paid amount reconciles to target paid amount after documented exclusions. Add relationship/orphan checks for reference joins.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

5. A fact join doubles revenue. Diagnose the likely grain/cardinality problem and prove the repair.

The fact likely joined to multiple dimension/history rows for one business key. State each input grain, measure join multiplicity before aggregation, constrain to one valid dimension row/version, rerun and prove fact key count plus revenue returns to the pre-join control.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

6. Explain why an incremental watermark using `max(updated_at)` can still miss data, and give a safer bounded strategy.

Timestamps can tie, arrive late or be corrected with older timestamps. A safer design may re-read $\geq$ boundary with unique-key merge, use a lookback window, source change sequence/CDC, partition rebuild or explicit version precedence. State the late-data limit honestly.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

7. Define a dbt-style staging/fact/dimension structure and where tests belong.

Raw sources feed thin deterministic staging models, then intermediate/business rules, then facts/dimensions at declared grains. Source tests validate inbound assumptions; staging tests protect normalized keys/types; mart tests protect uniqueness, relationships and business invariants.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

8. Describe an orchestration failure where progress outruns durable output; give the safe repair.

If an Airflow task writes output then fails before validation, or checkpoint commits before output, retry can duplicate or skip. Candidate progress should be computed first and committed only after durable output + validation. Failure must leave a replayable state.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

9. For Spark, explain how partition count, shuffle and skew evidence change a performance decision.

Establish result correctness, inspect partition sizes/explain/shuffle evidence, identify the real bottleneck, change one variable, then prove equivalent results. More partitions are not automatically faster; skew can dominate one task while averages look fine.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

10. Define Bronze/Silver/Gold as contracts rather than folder names.

Bronze preserves source evidence and ingestion metadata; Silver contains validated/deduplicated business structure; Gold serves declared consumer grain/semantics. Promotion requires explicit controls and reconciliation, not merely moving files between folders.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

11. An additive schema field appears. Describe compatibility, quarantine/versioning and downstream validation.

Classify the field as additive/compatible or breaking. Update explicit schema/contract deliberately, preserve raw evidence, quarantine unsupported records or version the contract, and prove downstream fields keep intended semantics. Never silently coerce a breaking change.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

12. Map a local lakehouse project to Fabric items without claiming a tenant run you did not execute.

Map data to OneLake/Lakehouse, transformations to notebooks/Spark, movement/orchestration to pipelines, serving to SQL endpoint/warehouse, and identities/secrets to least-privilege connections/Key Vault or managed identity. If no tenant run occurred, mark execution pending and keep only the validated local/design evidence.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

13. Choose CI checks that block deployment versus observability checks that alert after deployment.

Block merge/deploy on deterministic unit/data-contract/critical quality/security checks. Runtime observability monitors freshness, volume, error rate, latency, quality drift and ownership after deployment. Some signals can warn; critical invariants should block.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

14. Write the minimum contents of a data-pipeline runbook.

Prerequisites/start state, normal run, inputs/outputs/state locations, validation/reconciliation, common failures, recovery/replay/backfill, permissions/secrets, monitoring/alerts, cleanup, owner/escalation and known limits.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

15. Explain what a clean-clone proof establishes and what it does not establish.

Clean clone proves the documented repo can recreate the intended path without hidden local files/state. It does not prove production scale, cloud permissions, security hardening or every external failure mode unless those were actually tested.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

16. Give a two-minute technical defense structure for a portfolio project.

State the business problem and grain; draw the flow/state boundary; explain one key trade-off; show one correctness control; walk through one failure/recovery; show changed retry; finish with one known limitation and what you would measure next.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

17. Capstone incident: duplicated payment event plus stale inventory CDC change. Explain safe diagnosis, replay and uniqueness/version controls.

Preserve raw incident evidence. Payment uniqueness should use stable payment/event identity so duplicate replay is idempotent. Inventory applies monotonic source sequence/version so stale change cannot overwrite newer state. Stop progress on infrastructure failure, repair, replay and reconcile order/payment/inventory controls.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

18. State when Spark/Delta is justified in the capstone and when it is unnecessary complexity.

Use Spark/Delta when volume, distributed transforms, file/table semantics, incremental merge or replay justify it. For small daily data that PostgreSQL/Python/dbt handles comfortably, adding a cluster/lakehouse may increase cost and

operational burden without solving a requirement.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

19. Explain how least privilege and secret boundaries should appear in a Fabric/cloud deployment design.

Separate human and workload identities, grant only required workspace/item/data actions, avoid shared admins, keep secrets out of code/repos, prefer managed identity/short-lived credentials, and isolate PII access from general analytics serving.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.

20. List the evidence bundle you would show an interviewer to prove this is your own engineering work.

README + architecture/ADR, saved profiling, reproducible commands, expected controls, code/tests, reconciliation outputs, first failure and recovery, changed-input retry, clean-clone proof, runbook, screenshots/log snippets where real, and a clear list of what was not executed.

Validation: Compare your answer against the invariant, not just vocabulary. If you missed state ownership, reconciliation, failure behavior or evidence, repair that part and do a changed retry.